Security Log Parser Goal 3 Portfolio Report

Evidence for Foundational Skills in Software Development Tools

Field	Details
Student	Dongwoo Won
Student ID	25623683
Subject	UTS 43030 Professional Practice in Computing
Evidence goal	Goal 3 Demonstrate foundational skills in Software Development Tools
Project	Security Log Parser
Verification date	8 November 2026

This report documents a working, reproducible Python command-line project and the evidence produced by actual test and execution runs. It also identifies what remains for the student to complete personally.

Purpose and evidence conclusion

Security Log Parser provides concrete evidence for most project-based parts of Goal 3 in the supplied Professional Learning Plan. The package contains a standard-library Python application, a command-line interface, automated tests, fictional sample data, JSON and CSV output, reproducible instructions, safe-data guidance and actual execution records.

Fourteen automated tests passed in the working directory and again in a separate clean copy. The verified sample run processed eight valid events, counted two successful and six failed logins, skipped two malformed lines, and detected one user whose three failures occurred within an inclusive 10-minute window.

Requirement status

Requirement basis	Status	Evidence or qualification
Goal 3 areas in the Learning Plan	Verified	Mapped to source, tests, CLI workflow, documentation and clean reproduction in this package.
Working security log parser	Verified	Source and fictional inputs are included and actual runs completed successfully.
Tests and README	Verified	Fourteen unittest cases passed; README covers setup, use, expected output and limitations.
Git history authored by Dongwoo	Pending	No past or student-authored history was fabricated. A current Git workflow and student exercises are supplied.
Personal reflection	Pending	Prompts are included for Dongwoo to answer from direct experience.
Software Development Tools module requirements	Unverified	The Learning Plan refers to the module, but the module guide itself was not present in the supplied attachment.
Module completion and learning hours	Not claimed	This package contains no invented completion record or hours.

Source basis

The supplied Professional Learning Plan identifies Goal 3 as foundational skills in Software Development Tools. Page 2 lists Git-based version control and branching, Linux command-line workflow, debugging and unit testing, dependency and environment management, secure coding, README documentation and reproducible setup. It proposes a small Python security log-parser project and names working code, Git history, tests, a README and reflection as success evidence.

Project design and structure

The implementation separates data parsing, event analysis, export and command-line behavior. This makes each responsibility small enough to explain and test. The runtime uses only Python 3.9 or later and the standard library, so a package installation step is unnecessary.

Component	Responsibility
security_log_parser/parser.py	Reads UTF-8 CSV records, validates field count, timestamps and statuses, and records non-fatal warnings.
security_log_parser/analysis.py	Counts outcomes per user and applies the inclusive sliding-window failure rule.
security_log_parser/exporters.py	Writes a detailed JSON report and a per-user CSV summary.
security_log_parser/cli.py	Validates arguments, handles file errors, prints the summary and chooses output paths.
tests	Covers parsing, analysis, time boundaries, configuration, CLI errors and exports.
data	Provides fictional sample input and an empty input file.
evidence	Preserves raw command output, numbered captions and terminal-style captures.
README.md	Explains setup, commands, output, Git, debugging, limitations and safe handling.

Input and detection rules

- Each record contains a timezone-aware ISO 8601 timestamp, a fictional user, SUCCESS or FAILURE, and a fictional source.
- Blank lines, comments and the optional header are ignored. Invalid records are skipped and reported with their line number.
- Only FAILURE events enter detection. Events are grouped per user and sorted by time.
- The time boundary is inclusive. A span exactly equal to the configured window qualifies; a larger span does not.
- The tool reports the first qualifying window per user and exports the same result to JSON and CSV.

Verified execution results

Automated tests

The final working-directory run completed 14 tests with no failures. Coverage includes normal and empty inputs, malformed records, missing files, exports, counts, threshold behavior, exact and just-outside time boundaries, out-of-order events and invalid configuration.



Figure 1 Automated test run showing 14 passing tests

Debugging evidence

The first run failed because the initial data classes used the slots option, which is unavailable in Python 3.9. The fix removed that option and changed the documented minimum from Python 3.11 to Python 3.9. The original failed transcript remains in evidence/raw/debugging-first-run.txt. The successful test run followed the fix.

Sample analysis and error handling

The fictional sample contains eight valid events and two intentionally malformed lines. The program reported two successes and six failures. Sentinel-demo generated one alert because its three failures occurred at 09:10, 09:15 and 09:20, exactly filling the inclusive 10-minute boundary.

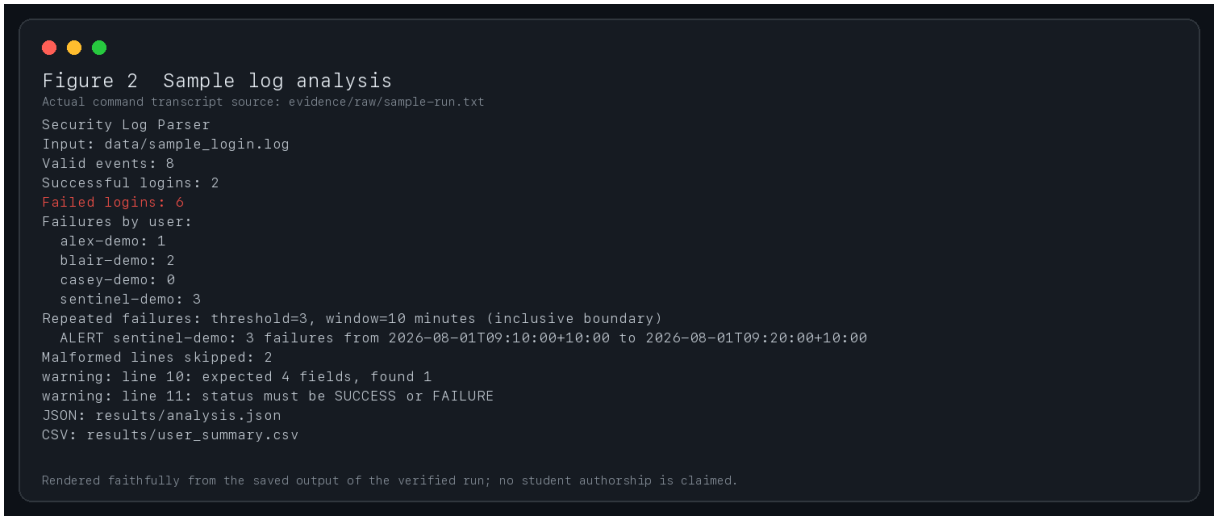


Figure 2 Sample log analysis with counts warnings and the inclusive boundary alert

Missing file behavior

A missing input file produces a concise message on standard error and exit status 2. This behavior can be detected by shell scripts and automated workflows.

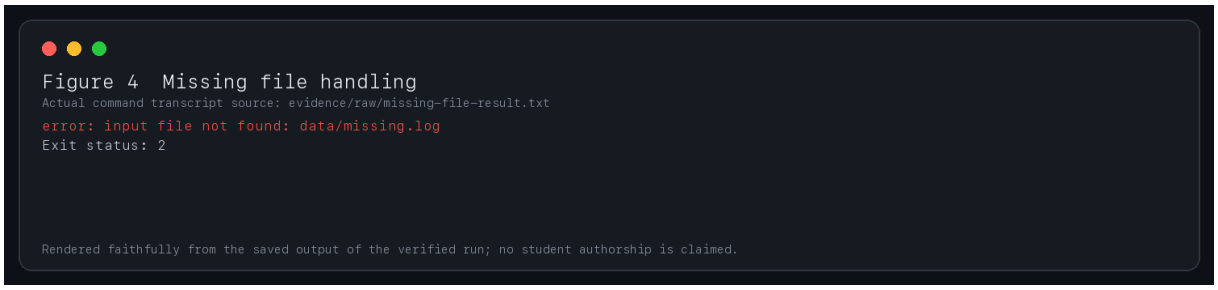


Figure 4 Missing file error and exit status 2

Exported evidence

The CSV contains one row per user with success and failure counts, the detection flag and the qualifying window. The JSON retains the configuration, totals, alerts and rejected-line details. The two formats support spreadsheet review and structured programmatic review.



Figure 3 Generated CSV and selected JSON fields

Clean environment reproduction

A clean project copy was created outside the working project without carrying over the generated results directory. Under Python 3.9.6, all 14 tests passed and the sample command regenerated analysis.json and user_summary.csv with the same summary and alert.

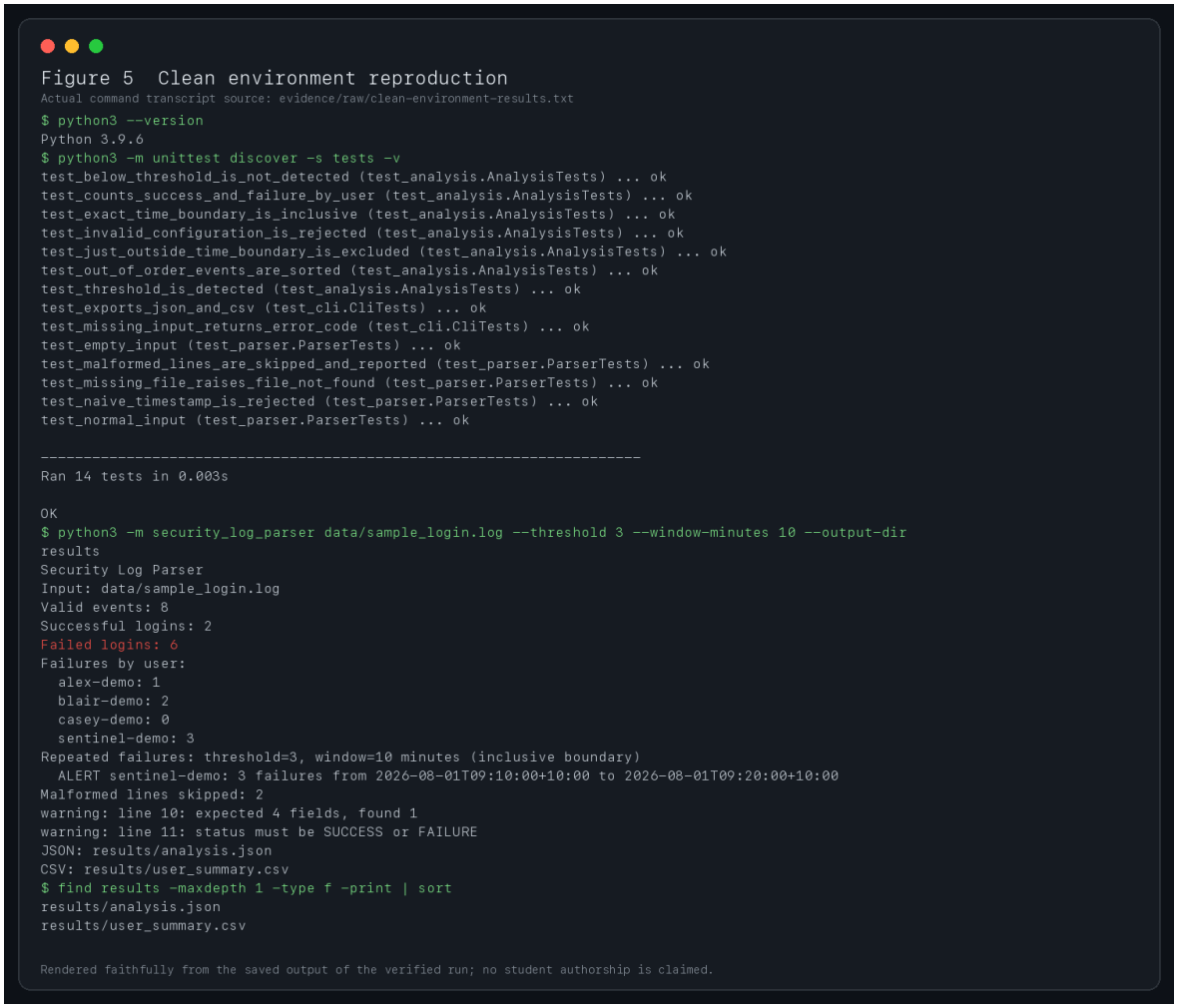


Figure 5 Clean copy test run sample analysis and regenerated output files

Connection to Goal 3

Learning Plan area	Evidence in this package	Status for portfolio use
Git version control and branching	README supplies an honest branch and commit workflow plus three scoped student changes.	Workflow documented. Student-authored Git evidence remains to be created.
Linux command-line workflow	Run, test, inspect, compare and exit-status commands are documented. Raw transcripts record actual AI-run commands.	Technical evidence present; student should repeat and explain the commands.
Debugging and unit testing	Initial Python 3.9 incompatibility was reproduced, diagnosed and fixed. Fourteen unittest cases pass.	Verified.
Dependency and environment management	No third-party runtime dependencies; Python version documented; clean-copy verification completed.	Verified for this project.
Secure coding	Strict schema and timestamp validation, non-execution of input, clear error handling and safe-data guidance.	Verified within the stated limitations.
README documentation	README covers setup, input format, commands, output, debugging, Git, safety and limitations.	Verified.
Reproducible setup	A separate clean copy passed all tests and produced the expected reports.	Verified.
Software Development Tools module	Only the Learning Plan reference was available; no module instructions or completion record were supplied.	Compliance and completion unverified.

Evidence interpretation

The package demonstrates that the technical artifacts work. It does not establish that Dongwoo personally performed the AI-run verification or completed the planned module. Strong final evidence should add student-authored Git commits, a student-run command record, answers to the comprehension questions, and reflection based on direct work with one of the modification exercises.

Limitations and safe data handling

Technical limitations

- The parser reads the file into memory and is not designed for continuous or very large streams.
- Only one four-field CSV schema and two status values are accepted.
- Detection is per user and reports the first qualifying window, not every overlapping window.
- Timestamps are validated syntactically but their trustworthiness is not verified.
- The alert is an educational indicator, not proof of malicious activity or a replacement for a security monitoring platform.
- Reports are unencrypted and files with the same output names are replaced.

Safe data handling

All supplied logs use fictional labels. Real authentication logs can contain names, device identifiers, IP addresses, locations and security-sensitive behavior. Before using real data, obtain authority, minimise fields, restrict access, follow retention rules, remove secrets and avoid committing logs or reports to Git. Rejected-line content also requires review before sharing because malformed records may still contain sensitive data.

Code safety decisions

- The program never evaluates or executes log content.
- Argument values are validated before analysis.
- Malformed records are isolated and reported instead of causing a partial crash.
- Timezone-aware timestamps reduce ambiguous boundary comparisons.
- There are no network calls or external runtime dependencies.

AI contribution and student responsibility

OpenAI Codex created the initial source code, tests, fictional sample data, README, evidence captures and report structure. Codex ran the recorded tests and commands, diagnosed the first compatibility failure, applied the fix and repeated the workflow in a clean copy.

Dongwoo Won should review and understand every submitted artifact, run the project independently, complete at least one modification exercise, create genuine current Git history under the correct identity, answer the comprehension questions, and write reflection responses from personal experience. The final submission should not imply that Dongwoo authored the AI-generated initial version or personally performed AI-run commands.

Claims excluded from this report

- No past Git dates or student-authored commit history were created.
- No learning hours were recorded as completed.
- No Software Development Tools module completion was claimed.
- No personal reflection was invented.
- No real login or personal data was used.

Reflection prompts for Dongwoo

1. Which command or file was hardest to understand when you ran the project, and what did you do to resolve that difficulty?
2. What failure did you encounter while completing your own modification, and how did a test or debugger help you locate it?
3. How did using a Git branch and reviewing the diff change the way you managed the modification?
4. What did the exact 10-minute boundary test teach you about translating a written rule into code?
5. Which secure-data practice would matter most if the fictional input were replaced with a real authentication log, and why?
6. What would you change in your next software project to make its setup and evidence more reproducible?

Response note: answer these prompts only after completing and recording the work yourself.

Student exercises and comprehension check

Code modification exercises

1. Add a --status-case-sensitive option. Test both default and strict behavior, then explain the compatibility trade-off.
2. Add a per-source failure summary to JSON and CSV. Use a feature branch and test one source shared by two users.
3. Add a --redact-usernames option using stable SHA-256-based aliases. Explain why unsalted hashing may still allow guessing and what a protected salt changes.

Comprehension questions

1. Why does the parser require a UTC offset in every timestamp?
2. What condition in the sliding-window logic makes the time boundary inclusive?
3. Why are malformed lines reported instead of silently ignored?
4. How does the analysis exclude successful logins from repeated-failure detection?
5. What is the difference between the threshold and the time window?
6. Why can a passing test suite still be insufficient evidence that the program is correct?
7. What does exit status 2 communicate to a shell script?
8. Which changes should be reviewed before a Git commit, and why?
9. What sensitive information could appear in real authentication logs even when passwords are absent?
10. Which limitation would you address first for a large production log, and what design would you consider?

Submission checklist

- Run the tests and sample command personally and retain the output.
- Complete one exercise with current, accurate Git commits.
- Answer the comprehension and reflection questions in your own words.
- Check that the final package still contains only fictional data.
- Confirm submission rules against the actual Software Development Tools module guide when it is available.